



Set Up Kubernetes Deployment

JJ

jjoseph1608@outlook.com

```
ec2-user@ip-172-31-4-157 ~]$ cd nextwork-flask-backend
(ec2-user@ip-172-31-4-157 ~)$ ls
Dockerfile  README.md  app.py  requirements.txt
(ec2-user@ip-172-31-4-157 ~)$ docker build -t nextwork-flask-backend .
[+] Building 12.0s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load metadata for docker.io/library/python:3.9-alpine
=> [internal] load .dockerignore
=> [internal] load build context
=> transferring context: 42.43kB
=> [1/5] FROM docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> resolve docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> sha256:ac5fec9ec2244c78ecbb822308adf80f8429f06aa1ef08f75572ffdb2124470d 5.18kB / 5.18kB
=> sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB
=> sha256:e9008197b8dd9d9b4c7a87e38fa66c8bceb4ff7d87df8e9061284b123459f9451 456.92kB / 456.92kB
=> sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d 15.40MB / 15.40MB
=> sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c 10.29kB / 10.29kB
=> sha256:a381ce006821bdce2a0d62e601890f912e9f23d9efb61007e58328863bd7708 1.73kB / 1.73kB
=> extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 0.1s
=> sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3 247B / 247B
=> extracting sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4ff7d87df8e9061284b123459f9451 0.1s
=> extracting sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d 0.9s
=> extracting sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3 0.0s
[2/5] WORKDIR /app
=> exporting layers 0.1s
=> [3/5] COPY requirements.txt requirements.txt 0.0s
=> [4/5] RUN pip3 install -r requirements.txt 8.1s
=> [5/5] COPY . . 0.1s
=> exporting to image 0.8s
=> writing image sha256:b6e1ed9912dcbcc57576217a058c9a04393f92d1a061375e185a2530990ac9eab 0.8s
=> naming to docker.io/library/nextwork-flask-backend 0.0s
(ec2-user@ip-172-31-4-157 ~)$
```



Introducing Today's Project!

In this project, I will take a backend application from GitHub, build a Docker image of it, and push that image to Amazon ECR. I'm doing this to learn how to containerize applications and get them ready to deploy to my Kubernetes cluster.

Tools and concepts

I launched an EC2 instance and used `eksctl` to create an EKS cluster. Then I cloned the backend code from GitHub, explored the three files that make it up (Dockerfile, requirements.txt, and app.py), and built a Docker image from it. I hit a permissions error because Docker needed root access but I was logged in as `ec2-user`. I fixed that by adding `ec2-user` to the Docker group. Finally I pushed the image to Amazon ECR so my EKS cluster can pull it and deploy it

Project reflection

This project took me around 60 minutes to complete. The most challenging part was dealing with the Docker permissions error. I had to figure out that Docker was installed for the root user while I was logged in as `ec2-user`. Once I realized I needed to add `ec2-user` to the Docker group, things moved faster. My favorite part was exploring the backend code on GitHub. It was cool to see exactly what I was containerizing. Understanding how the `app.py` file connects to the Hacker News API and returns JSON made the whole process feel less abstract. Instead of just pushing some mystery image to ECR, I knew what was actually running inside that container

placeholder

Something new that I learnt from this experience was how Docker permissions work. I'd never dealt with the difference between root access and regular user access before.



jjoseph1608@outlook....

NextWork Student

nextwork.org

When I ran into the ec2-user problem, I had to understand that Docker needs special privileges to run, and you can't just sudo your way around it permanently. You need to actually grant the user group permission. That taught me that sometimes the quick fix isn't the right fix, and taking time to do it properly saves you from typing sudo over and over. I also learned that a Dockerfile is really just instructions to the Docker engine. It's not magic. You're telling Docker to start with a base image, install packages, copy files, and set a command to run. Understanding those three pieces of my backend code made the whole containerization process feel less abstract. I wasn't just pushing some mystery container. I knew exactly what was inside it



What I'm deploying

To set up today's project, I launched a Kubernetes cluster. Steps I took to do this included launching an EC2 instance and connecting to it using EC2 Instance Connect. Then I installed eksctl on the instance so I could create the cluster from the command line. I configured my AWS credentials by running `aws configure` and entering my access key and secret key. After that I ran the `eksctl create cluster` command with parameters for the cluster name, node type, number of nodes, Kubernetes version, and region. I watched the cluster get built through CloudFormation while it created the VPC, security groups, IAM roles, and EC2 instances for the worker nodes. Once the cluster was running, I opened the EKS console and set up an IAM access entry for myself so I could actually interact with it. That last part was crucial because without the access entry, the cluster wouldn't recognize me as someone allowed to run commands against it

I'm deploying an app's backend

Next, I retrieved the backend that I plan to deploy. An app's backend means the server side code that runs the actual business logic, handles database queries, and processes requests from the frontend. The frontend is what users see and interact with, but the backend does all the heavy lifting behind the scenes. I retrieved backend code by running `git clone` followed by the GitHub repository URL on my EC2 instance. Git downloaded the entire project to my instance so I could work with it locally. Once it was cloned, I could look at the code, build a Docker image from it, and push that image to Amazon ECR



jjoseph1608@outlook....

NextWork Student

nextwork.org

```
[ec2-user@ip-172-31-4-157 ~]$ ls
nextwork-flask-backend
[ec2-user@ip-172-31-4-157 ~]$
```



Building a container image

Once I cloned the backend code, my next step is to build a container image of the backend. This is because a container image packages the application with all its dependencies so it can run the same way everywhere. Once I have the image, I can push it to Amazon ECR and then deploy it to my Kubernetes cluster where it will actually run

When I tried to build a Docker image of the backend, I ran into a permissions error because Docker was installed for the root user, but I was logged in as ec2-user. The Docker engine needs root level access to run, so when ec2-user tries to run Docker commands, it gets blocked

To solve the permissions error, I added ec2-user to the Docker group so it could run Docker commands without needing sudo every time. The Docker group is a Linux group that has permission to access the Docker engine. When you add a user to the Docker group, that user can run Docker commands directly without needing root access through sudo. This is cleaner than using sudo for every Docker command



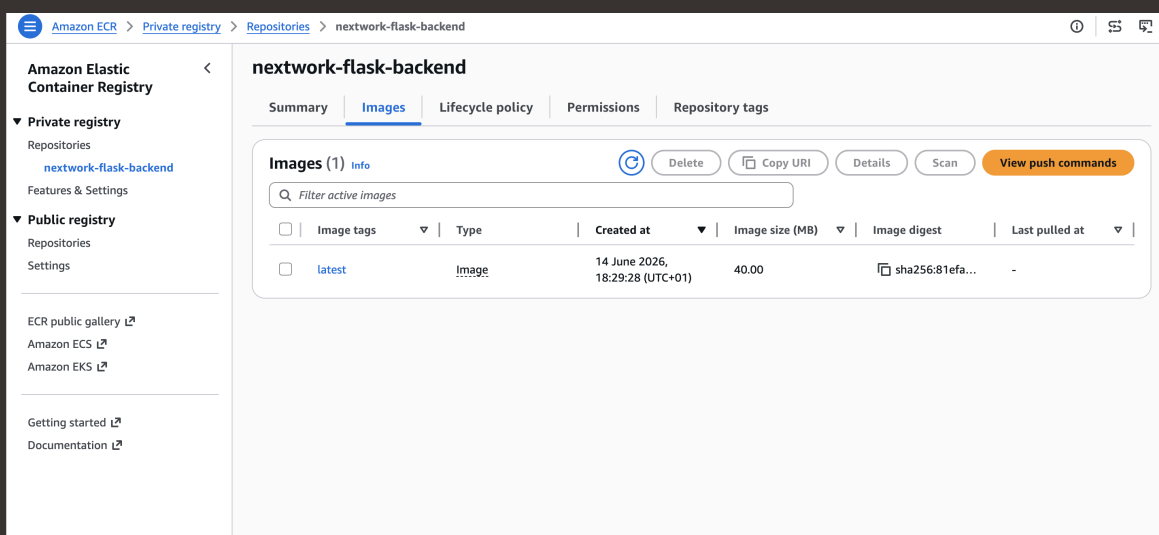
```
[ec2-user@ip-172-31-4-157 ~]$ cd nextwork-flask-backend
[ec2-user@ip-172-31-4-157 nextwork-flask-backend]$ ls
Dockerfile  README.md  app.py  requirements.txt
[ec2-user@ip-172-31-4-157 nextwork-flask-backend]$ docker build -t nextwork-flask-backend .
(*) Building 12.0s (10/10) FINISHING
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 269B
=> [internal] load metadata for docker.io/library/python:3.9-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 42.43kB
=> [1/5] FROM docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> => resolve docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> => sha256:ac5fec9ec2244c78ecbb822308adf80f8429fc6aa1ef08f75572ffdb2124470d 5.18kB / 5.18kB
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5d5d5d 3.80MB / 3.80MB
=> => sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451 456.92kB / 456.92kB
=> => sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d 15.40MB / 15.40MB
=> => sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c 10.29kB / 10.29kB
=> => sha256:a381ce006821bdce2a0d62e601890f912e9f23d9e1fb51007e58328863bd7708 1.73kB / 1.73kB
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5d5d5d
=> => sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3 247B / 247B
=> => extracting sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451
=> => extracting sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d
=> => extracting sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt requirements.txt
=> [4/5] RUN pip3 install -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:beled9912dcbcc57576217a058c9a04393f92d1a061375e185a2530990ac9eab
=> => naming to docker.io/library/nextwork-flask-backend
[ec2-user@ip-172-31-4-157 nextwork-flask-backend]$
```



Container Registry

I'm using Amazon ECR in this project to store the Docker image I just built so my Kubernetes cluster can pull it and deploy it. ECR is a good choice for the job because it's integrated with AWS, which means my EKS cluster can access the images without any extra configuration or authentication hassle

Container registries like Amazon ECR are great for Kubernetes deployment because they give you a central place to store Docker images that your cluster can pull from whenever it needs to deploy something. Instead of rebuilding images every time you want to run your application, you build once, push to ECR, and then any Kubernetes cluster can grab that image and run it. You can version your images so you know exactly which version is running in production. If you need to roll back to an older version, it's right there in ECR. ECR also integrates directly with EKS, so your cluster doesn't need extra authentication or configuration to pull images. It just works. And if you have multiple clusters or teams, they can all use the same images from the same registry instead of everyone maintaining their own copy





EXTRA: Backend Explained

After reviewing the app's backend code, I learned that my backend is actually pretty straightforward. Flask sets up the routes and handles incoming requests. When someone visits the `/contents/[topic]` endpoint, the `SearchContents` class runs, grabs that topic from the URL, and makes a request to the Hacker News Search API. The code pulls specific details like the id, title, and url from the Hacker News results, then formats everything into JSON and sends it back. So my backend isn't doing complex calculations or business logic. It's mostly acting as a middleman between the user and the Hacker News API, translating requests and formatting responses. This matters because when I containerize this application, Docker just needs to install Flask and Python, copy the code into the container, and make sure the application can reach the Hacker News API. There's nothing fancy to configure

Unpacking three key backend files

The `requirements.txt` file lists all the Python packages your application needs to run. When you install packages with pip, you can save them to this file so anyone who wants to run your code can install the same versions with one command. Instead of having to manually track which packages you used and what versions, you just run ``pip install -r requirements.txt`` and Python installs everything listed. This matters for Docker because when you build your image, you include the `requirements.txt` file and install all the dependencies during the build process. That way your container has everything it needs to run your backend code

The Dockerfile gives Docker the instructions it needs to build your image. It starts with a base image, then runs commands to install dependencies, copy your code, and set up how the container should run. Key commands in this Dockerfile include `FROM`, which picks the base image your container starts with. `RUN` executes commands during the build, like installing packages with pip. `COPY` takes files from your EC2 instance and puts them inside the container. `EXPOSE` tells Docker which port your application listens on. `CMD` specifies what



app.py contains my backend's main code. It does three things. First, it sets up Flask and creates routes. The /contents/<topic> route takes incoming requests and sends them to the SearchContents class. Second, it fetches data. When someone visits my backend, the SearchContents class pulls the topic from the URL, makes a request to the Hacker News Search API, and collects details like id, title, and url. Third, it sends back JSON. My backend processes the data and returns it in a format other apps can read. An API is how different programs talk to each other. My backend uses the Hacker News API to grab content. My backend itself is also an API. It accepts requests, gets data from Hacker News, and sends responses back. This lets my frontend or other services get data from my backend



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

